LAPORAN PRAKTIKUM

STRUKTUR DATA

“Stack”

disusun Oleh:

Ardra Athailla Firsy

2511533028

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T

Asisten Praktikum:

Sherly Sukmadira

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Assalamu'alaikum Warahmatullahi Wabarakatuh

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum *Struktur Data* ini dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu syarat untuk memenuhi tugas praktikum pada mata kuliah Struktur Data. Dalam laporan ini, penulis memaparkan hasil praktikum yang telah dilakukan, mulai dari konsep dasar hingga implementasi struktur data seperti array, stack, queue, dan linked list dalam pemrograman.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat banyak kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun dari pembaca demi perbaikan di masa yang akan datang.

Tidak lupa, penulis mengucapkan terima kasih kepada dosen pengampu dan semua pihak yang telah membantu dalam pelaksanaan praktikum serta penyusunan laporan ini.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat dan menambah wawasan bagi pembaca, khususnya dalam memahami konsep dan penerapan struktur data.

Padang, 9 April 2025

Ardra Athailla Firsy

DAFTAR ISI

KATA PENGANTAR ii

DAFTAR ISI iii

BAB I PENDAHULUAN 1

1.1 Latar Belakang 1

1.2 Tujuan 1

1.3 Manfaat 1

BAB II PEMBAHASAN 3

2.1 Pengertian Stack 3

2.2 Implementasi Stack pada Java 3

2.2.1 Class Nilai Maksimum (NilaiMaksimum_2511533028) 4

2.2.2 Class Siswa Stack (SiswaStack_2511533028) 5

2.2.3 Class Stack Array (StackArray_2511533028) 7

2.2.4 Class Stack Array Driver (StackArrayDriver_2511533028) 8

2.2.5 Class Stack Post fix (StackPostfix_2511533028) 9

BAB III PENUTUP 11

3.1 Kesimpulan 11

3.2 Saran 11

DAFTAR PUSTAKA 12

LAPORAN TUGAS PRAKTIKUM 13

1. Class Website (Website_2511533028) 13

2. Class Browser (Browser_2511533028) 14

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan perangkat lunak, pengelolaan data secara efisien menjadi hal yang sangat penting, terutama ketika berhubungan dengan proses yang bersifat berurutan. Salah satu struktur data yang banyak digunakan untuk menangani kondisi tersebut adalah **Stack**. Stack merupakan struktur data yang bekerja dengan prinsip **LIFO (Last In First Out)**, yaitu elemen terakhir yang masuk akan menjadi elemen pertama yang keluar. Konsep ini banyak digunakan dalam berbagai aplikasi seperti manajemen memori, pemanggilan fungsi (call stack), undo-redo pada aplikasi, serta evaluasi ekspresi seperti postfix dan prefix. Oleh karena itu, pemahaman tentang stack sangat penting bagi mahasiswa atau pengembang agar dapat mengimplementasikan solusi yang tepat dalam pemrograman, khususnya dalam menangani data yang membutuhkan pengolahan secara bertumpuk.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum pekan 3 tentang Stack adalah sebagai berikut:

1. Memahami konsep dasar struktur data Stack dan prinsip kerjanya (LIFO).
2. Mengetahui operasi dasar pada Stack seperti push, pop, dan peek.
3. Mampu mengimplementasikan Stack menggunakan berbagai cara seperti array maupun ArrayList.
4. Melatih kemampuan logika dan algoritma dalam pemrograman.

1.3 Manfaat

Manfaat yang diharapkan dari pelaksanaan praktikum pekan 3 tentang Stack adalah sebagai berikut:

1. Membantu dalam mengelola data yang bersifat bertumpuk secara efisien.
2. Mempermudah implementasi fitur penting seperti **undo/redo**, navigasi (back/forward), dan call stack pada program.
3. Digunakan dalam proses parsing dan evaluasi ekspresi dalam compiler atau kalkulator.
4. Meningkatkan pemahaman tentang struktur data lain yang lebih kompleks.
5. Membantu dalam menyelesaikan berbagai permasalahan algoritma yang membutuhkan urutan khusus dalam pengolahan data.

BAB II
PEMBAHASAN

2.1 Pengertian Stack

Stack adalah salah satu struktur data linear yang bekerja berdasarkan prinsip **LIFO (Last In First Out)**, yaitu elemen terakhir yang dimasukkan ke dalam stack akan menjadi elemen pertama yang dikeluarkan. Dalam implementasinya, stack hanya memiliki satu titik akses utama yang disebut **top**, yaitu posisi tempat operasi penambahan (**push**) dan penghapusan (**pop**) dilakukan. Selain itu, terdapat juga operasi **peek** yang digunakan untuk melihat elemen teratas tanpa menghapusnya. Stack dapat diimplementasikan menggunakan array maupun struktur data dinamis seperti linked list atau ArrayList. Struktur data ini banyak digunakan dalam berbagai aplikasi seperti manajemen memori (call stack), evaluasi ekspresi matematika (postfix dan prefix), parsing pada compiler, serta fitur undo dan redo pada perangkat lunak.

2.2 Implementasi Stack pada Java

Pada praktikum pekan 3 mahasiswa disuruh untuk mengikuti intruksi dari dosen. Yaitu menuliskan kembali kode program yang sudah ada di layar TV. Kode program yang dibuat tentang Array List, di bawah ini kode program yang dituliskan pada praktikum pekan 3.

2.2.1 Class Nilai Maksimum (NilaiMaksimum_2511533028)

```java
package pekan3_2511533028;

import java.util.Stack;

public class NilaiMaksimum_2511533028 {
    public static int max(Stack<Integer> s) {
        Stack<Integer> backup = new Stack<Integer>();
        int maxValue = s.pop();
        backup.push(maxValue);
        while (!s.isEmpty()) {
            int next = s.pop();
            backup.push(next);
            maxValue = Math.max(maxValue, next);
        }
        while (!backup.isEmpty()) {
            s.push(backup.pop());
        }
        return maxValue;
    }
    public static void main(String[] args) {
        Stack<Integer> s = new Stack<Integer>();
        s.push(70);
        s.push(12);
        s.push(20);
        System.out.println("isi stack "+s);
        System.out.println("Stack Teratas "+s.peek());
        System.out.println("Nilai maksimum "+max(s));
    }

}
```

```
isi stack [70, 21, 20]
Stack Teratas 20
Nilai maksimum 70
```

Gambar 2.2.1

Kode program di atas merupakan program Java yang menggunakan struktur data **Stack** untuk mencari nilai maksimum tanpa menghilangkan isi stack aslinya. Pada bagian awal, package Pekan3_2511533028; menunjukkan lokasi package dari kelas tersebut, dan import java.util.Stack; digunakan untuk mengimpor kelas Stack dari library Java. Kelas NilaiMaksimum_2511533028 berisi method utama max_3028 yang bertipe int dan menerima parameter berupa Stack<Integer> s_3028, dimana fungsi ini bertujuan mencari nilai terbesar dalam stack. Di dalam method ini dibuat stack baru bernama backup_3028 yang berfungsi sebagai tempat penyimpanan sementara agar data stack asli tidak hilang, kemudian elemen pertama diambil menggunakan pop() dan disimpan sebagai nilai awal maksimum (maxValue_3028), lalu dimasukkan ke stack backup. Selanjutnya dilakukan perulangan while selama stack s_3028belum kosong, setiap elemen di-pop, dimasukkan ke backup, dan dibandingkan dengan nilai maksimum menggunakan Math.max() untuk memperbarui nilai maksimum. Setelah semua

elemen diproses, dilakukan perulangan kedua untuk mengembalikan isi stack dari backup_3028 ke s_3028 sehingga kondisi awal stack tetap terjaga. Method ini kemudian mengembalikan nilai maksimum yang ditemukan. Pada method main, dibuat objek stack s_3028 lalu ditambahkan beberapa nilai menggunakan push() yaitu 70, 21, dan 20, kemudian isi stack ditampilkan, elemen teratas ditampilkan menggunakan peek(), dan terakhir nilai maksimum dari stack ditampilkan dengan memanggil method max_3028(s_3028).

2.2.2 Class Siswa Stack (SiswaStack_2511533028)

```java
package pekan3_2511533028;

import java.util.ArrayList;

    class Siswa_2511533028 {
        String nama;
        int nim;

        public Siswa_2511533028(String nama, int nim) {
            this.nama = nama;
            this.nim = nim;
        }

        @Override
        public String toString() {
            return "Nim: " + nim + ", Nama: " + nama;
        }
    }
    public class SiswaStack_2511533028 {
        private ArrayList<Siswa_2511533028> stack;

        public SiswaStack_2511533028() {
            stack = new ArrayList<>();
        }

        public void push(Siswa_2511533028 mhs) {
            stack.add(mhs);
        }

        public Siswa_2511533028 pop() {
            if (!isEmpty()) {
                return stack.remove(stack.size() - 1);
            }
            return null;
        }

        public Siswa_2511533028 peek() {
            if (!isEmpty()) {
                return stack.get(stack.size() - 1);
            }
        return null;
    }

    public boolean isEmpty() {
        return stack.isEmpty();
    }

    public void tampilkanSiswa() {
        for (int i = stack.size() - 1; i >= 0; i--) {
            System.out.println(stack.get(i));
        }
    }
    public static void main(String[] args) {
        SiswaStack_2511533028 studentStack = new SiswaStack_2511533028();

        Siswa_2511533028 mhs1 = new Siswa_2511533028("Ali", 1);
        Siswa_2511533028 mhs2 = new Siswa_2511533028("Boby", 2);
        Siswa_2511533028 mhs3 = new Siswa_2511533028("Charles", 3);

        studentStack.push(mhs1);
        studentStack.push(mhs2);
        studentStack.push(mhs3);

        System.out.println("Siswa di dalam stack:");
        studentStack.tampilkanSiswa();

        System.out.println("siswa teratas " + studentStack.peek());
        System.out.println("mengeluarkan siswa teratas dari stack: " + studentStack.pop());
        System.out.println("daftar siswa setelah di pop :");
        studentStack.tampilkanSiswa();
    }
}
```

```
Siswa di dalam stack:
Nim: 3, Nama: Charles
Nim: 2, Nama: Boby
Nim: 1, Nama: Ali
siswa teratasNim: 3, Nama: Charles
Mengeluarkan siswa teratas dari stack: Nim: 3, Nama: Charles
Mendaftar siswa setelah di pop :
Nim: 2, Nama: Boby
Nim: 1, Nama: Ali
```

Gambar 2.2.2

Kode program pada gambar tersebut mengimplementasikan struktur data **Stack berbasis ArrayList** untuk menyimpan objek mahasiswa. Pada bagian awal terdapat package Pekan3_2511533028; yang menunjukkan lokasi package, serta import java.util.ArrayList; untuk menggunakan struktur data dinamis ArrayList. Kelas Siswa_2511533028 berfungsi sebagai representasi data mahasiswa dengan atribut nama_3028(String) dan nim_3028 (int), yang diinisialisasi melalui konstruktor dengan keyword this untuk membedakan antara variabel parameter dan atribut kelas. Selain itu, method toString() di-override agar ketika objek ditampilkan, output-nya berupa format yang lebih informatif yaitu berisi NIM dan Nama mahasiswa. Selanjutnya terdapat kelas SiswaStack_2511533028 yang bertindak sebagai implementasi stack menggunakan

ArrayList<Siswa_2511533028> bernama stack_3028, dimana konstruktor kelas ini menginisialisasi ArrayList kosong. Method push() digunakan untuk menambahkan objek mahasiswa ke dalam stack dengan cara add() ke bagian akhir list, sedangkan method pop() digunakan untuk menghapus sekaligus mengembalikan elemen terakhir (top) dari stack dengan remove(size - 1) setelah memastikan stack tidak kosong menggunakan isEmpty(). Method peek() berfungsi untuk melihat elemen teratas tanpa menghapusnya dengan get(size - 1), dan method isEmpty() digunakan untuk mengecek apakah stack kosong dengan memanfaatkan method bawaan ArrayList. Method tampilkanSiswa_2511533028() digunakan untuk menampilkan seluruh isi stack dari atas ke bawah dengan perulangan dari indeks terakhir ke indeks pertama sehingga sesuai konsep LIFO. Pada method main, dibuat objek stack studentStack_3028, kemudian dibuat tiga objek mahasiswa (Ali, Boby, dan Charles) yang dimasukkan ke dalam stack menggunakan push(), lalu program menampilkan seluruh isi stack, menampilkan elemen teratas menggunakan peek(), menghapus elemen teratas menggunakan pop(), dan akhirnya menampilkan kembali isi stack setelah terjadi perubahan, sehingga memperlihatkan cara kerja stack secara lengkap mulai dari penambahan, pengambilan, hingga penghapusan data.

2.2.3 Class Stack Array (StackArray_2511533028)

```java
package pekan3_2511533028;

public class stackArray_2511533028 {
    static final int MAX = 1000;
    int top;
    int a[] = new int[MAX];
    boolean isEmpty()
    {
        return (top < 0);
    }
    stackArray_2511533028()
    {
        top = -1;
    }
    boolean push_2511533028(int x)
    {
        if (top >= (MAX - 1)) {
            System.out.println("Stack Overflow");
            return false;
        }
        else {
            a[++top] = x;
            System.out.println(x + " dimasukan dalam stack");
            return true;
        }
    }
    int pop_2511533028()
    {
        if (top < 0) {
            System.out.println("Stack Underflow");
            return 0;
        }
        else {
            int x = a[top--];
            return x;
        }
    }
    int peek()
    {
        if (top < 0) {
            System.out.println("Stack Underflow");
            return 0;
        }
        else {
            int x = a[top];
            return x;
        }
    }
    void print() {
    for(int i = top;i>-1;i--) {
      System.out.println(" "+ a[i]);
    }
  }
}
```

Gambar 2.2.3

Kode program tersebut merupakan implementasi struktur data **Stack berbasis array** dalam bahasa Java yang dirancang untuk menyimpan data bertipe integer dengan kapasitas maksimum tertentu. Pada bagian awal, package Pekan3_2511533028; menunjukkan bahwa kelas ini berada dalam package tertentu, kemudian didefinisikan kelas StackArray_2511533028 yang memiliki konstanta MAX_3028 = 1000 sebagai batas maksimal ukuran stack. Variabel top_3028 digunakan sebagai penunjuk posisi elemen teratas dalam stack, sedangkan array a_3028[] berfungsi sebagai media penyimpanan data. Method isEmpty_3028() digunakan untuk mengecek apakah stack dalam kondisi kosong dengan cara memeriksa apakah nilai top_3028 kurang dari 0. Konstruktor StackArray_2511533028() berfungsi untuk menginisialisasi nilai top_3028 menjadi -1 yang menandakan bahwa stack belum berisi elemen. Method push(int

x_3028) digunakan untuk menambahkan elemen ke dalam stack, dimana program terlebih dahulu mengecek apakah stack sudah penuh (overflow) dengan membandingkan nilai top_3028 dengan kapasitas maksimum, jika belum penuh maka nilai top_3028 akan dinaikkan terlebih dahulu kemudian elemen dimasukkan ke dalam array dan ditampilkan pesan bahwa data berhasil dimasukkan. Method pop_3028() berfungsi untuk menghapus dan mengambil elemen teratas dari stack, dengan pengecekan kondisi apakah stack kosong (underflow), jika tidak kosong maka elemen pada posisi top_3028 diambil lalu nilai top_3028 dikurangi satu untuk menghapus elemen tersebut dari stack. Method peek_3028() digunakan untuk melihat elemen teratas tanpa menghapusnya, dengan cara mengembalikan nilai pada indeks top_3028 setelah memastikan stack tidak kosong. Method print_3028() digunakan untuk menampilkan seluruh isi stack dari atas ke bawah dengan menggunakan perulangan dari indeks top_3028 hingga 0 sehingga sesuai dengan prinsip LIFO. Terakhir, method main disediakan sebagai titik awal eksekusi program, namun pada kode ini belum diisi sehingga belum ada proses yang dijalankan.

2.2.4 Class Stack Array Driver (StackArrayDriver_2511533028)

```java
package pekan3_2511533028;

public class stackArrayDriver_2511533028 {
    public static void main(String[] args) {
        stackArray_2511533028 s = new stackArray_2511533028();
        s.push_2511533028(10);
        s.push_2511533028(20);
        s.push_2511533028(30);
        System.out.println(s.pop_2511533028() + " dikeluarkan dari stack");
        System.out.println("Elemen teratas adalah :" + s.peek());
        System.out.print("Element pada stack");
        s.print();
    }

}
```

```
10 dimasukan dalam stack
20 dimasukan dalam stack
30 dimasukan dalam stack
30 dikeluarkan dari stack
Elemen teratas adalah :20
Element pada stack 20
 10
```

Gambar 2.2.4

Kode program tersebut merupakan **kelas driver (program utama)** yang digunakan untuk menguji implementasi stack berbasis array pada kelas StackArray_2511533028. Pada bagian main, pertama dibuat objek stack bernama s_3028 dari kelas StackArray_2511533028, yang berarti stack kosong siap digunakan. Selanjutnya dilakukan beberapa operasi push() yaitu memasukkan nilai 10, 20, dan 30 ke dalam stack secara berurutan, dimana sesuai konsep **LIFO (Last In First Out)** maka elemen terakhir yang masuk (30) akan berada di posisi teratas. Setelah itu, method pop_3028() dipanggil untuk menghapus dan mengambil elemen teratas dari stack, sehingga nilai 30 akan dikeluarkan dan ditampilkan dengan pesan bahwa elemen tersebut telah di-pop dari stack. Kemudian method peek_3028() digunakan untuk melihat elemen teratas yang baru tanpa menghapusnya, yang dalam kondisi ini adalah 20 karena 30 sudah dikeluarkan sebelumnya. Terakhir, method print_3028() dipanggil untuk menampilkan seluruh isi stack yang tersisa, yaitu 20 dan 10, ditampilkan dari atas ke bawah sesuai urutan stack. Secara keseluruhan, kode ini memperlihatkan cara kerja operasi dasar stack yaitu push, pop, peek, dan print dalam satu alur eksekusi program.

2.2.5 Class Stack Post fix (StackPostfix_2511533028)

```java
package pekan3_2511533028;

import java.util.Scanner;
import java.util.Stack;

public class StackPositfix_2511533028 {
    public static int postfixEvaluate(String expression) {
        Stack<Integer> s = new Stack<Integer>();
        Scanner input = new Scanner(expression);
        while (input.hasNext()) {
            if (input.hasNextInt()) {    // an operand (integer)
                s.push(input.nextInt());
            } else {
                String operator = input.next();
                int operand2 = s.pop();
                int operand1 = s.pop();
                if (operator.equals("+")) {
                    s.push(operand1 + operand2);
                } else if (operator.equals("-")) {
                    s.push(operand1 - operand2);
                } else if (operator.equals("*")) {
                    s.push(operand1 * operand2);
                } else {
                    s.push(operand1 / operand2);
                }
                }

            }
        input.close();
        return s.pop();
        }
    public static void main(String[] args) {
        System.out.println("hasil postfix= "+postfixEvaluate("5 2 4 * + 7 -"));
    }
}
```

```
Hasil postfix = 6
```

Gambar 2.2.5

Kode program tersebut merupakan implementasi **evaluasi ekspresi postfix (Reverse Polish Notation)** menggunakan struktur data **Stack** di Java. Pada bagian awal, package Pekan3_2511533028; menunjukkan lokasi package, serta import java.util.Scanner; dan import java.util.Stack; digunakan untuk membaca input string dan memanfaatkan struktur data stack bawaan Java. Kelas StackPostfix_2511533028 memiliki method utama postfixEvaluate(String expression) yang bertipe int dan berfungsi untuk menghitung hasil dari ekspresi postfix yang diberikan dalam bentuk string. Di dalam method ini dibuat objek Stack<Integer> s_3028 sebagai penampung operand (angka), serta objek Scanner input_3028 yang digunakan untuk membaca token (angka atau operator) dari string ekspresi. Proses utama dilakukan dengan perulangan while (input_3028.hasNext()), dimana jika token yang dibaca berupa bilangan (hasNextInt()), maka langsung dimasukkan ke dalam stack menggunakan push(), sedangkan jika token berupa operator, maka dua elemen teratas diambil dari stack menggunakan pop() sebagai operand2_3028 dan operand1_3028, lalu dilakukan operasi sesuai operator (+, -, *, atau /) dan hasilnya dimasukkan kembali ke dalam stack. Proses ini berlanjut hingga seluruh token selesai diproses, kemudian Scanner ditutup dan hasil akhir diambil dari stack dengan pop() sebagai nilai yang dikembalikan. Pada method main, program langsung memanggil postfixEvaluate("5 2 4 * + 7 -"), yang berarti menghitung ekspresi postfix tersebut, dan hasilnya ditampilkan ke layar. Secara keseluruhan, kode ini menunjukkan cara kerja evaluasi postfix yang memanfaatkan stack untuk menyimpan operand sementara dan melakukan operasi secara berurutan sesuai aturan postfix.

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan pembahasan mengenai struktur data **Stack** dan implementasi kode program yang telah dibuat, dapat disimpulkan bahwa stack merupakan struktur data linear yang bekerja dengan prinsip **LIFO (Last In First Out)**, dimana elemen terakhir yang masuk akan menjadi elemen pertama yang keluar. Stack memiliki operasi dasar seperti **push, pop, dan peek** yang digunakan untuk mengelola data secara terstruktur melalui satu titik akses yaitu **top**. Dalam implementasinya, stack dapat dibuat menggunakan berbagai metode seperti array maupun struktur data dinamis. Pada kode program yang dibahas, stack diimplementasikan menggunakan array dengan kapasitas tetap, dimana variabel top berperan penting dalam menentukan posisi elemen teratas. Program tersebut juga telah mencakup penanganan kondisi **overflow** (stack penuh) dan **underflow** (stack kosong), serta menyediakan fungsi untuk menampilkan isi stack. Secara keseluruhan, baik secara konsep maupun implementasi, stack merupakan struktur data yang sederhana namun sangat penting dan banyak digunakan dalam berbagai aplikasi pemrograman.

3.2 Saran

Adapun saran yang dapat diberikan berdasarkan materi ini adalah sebagai berikut:

1. Kembangkan implementasi stack menggunakan struktur data dinamis seperti Linked List atau ArrayList agar tidak terbatas kapasitas.
2. Tambahkan isi pada method main sebagai contoh penggunaan program agar dapat langsung diuji dan dipahami.
3. Gunakan penamaan variabel yang lebih sederhana dan konsisten supaya kode lebih mudah dibaca.
4. Kembangkan program dengan fitur tambahan seperti pencarian data dalam stack atau penghitungan tertentu.

DAFTAR PUSTAKA

[1] T. H. Cormen, C. E. Leiserson dan R. L. Rivest, “Introducing to Algorithms,” 2009. [Online].

[2] M. T. Goodrich, R. Tamassia dan M. H. Goldwasser, “Data Struchtures and Algorithms in Java,” 2014. [Online].

LAPORAN TUGAS

PRAKTIKUM

SOAL:

Pada pekan ini, mahasiswa diminta untuk mengimplementasikan konsep Stack (Tumpukan) yang bekerja dengan prinsip LIFO (Last In, First Out). Tema tugas kali ini adalah Simulasi Browser History. Mahasiswa diwajibkan membuat program yang mensimulasikan perpindahan halaman web. Halaman yang terakhir dibuka akan berada di posisi teratas tumpukan, dan menjadi halaman pertama yang dihapus saat tombol "Back" ditekan.

1. Class Website (Website_2511533028)

```java
package tugaspekan3_2511533028;

public class Website_2511533028 {
    private String judul_3028;
        private String url_3028;

        // Constructor
        public Website_2511533028(String judul_3028, String url_3028) {
            this.judul_3028 = judul_3028;
            this.url_3028 = url_3028;
        }

        // Getter
        public String getJudul_3028() {
            return judul_3028;
        }

        public String getUrl_3028() {
            return url_3028;
        }

        // Setter
        public void setJudul_3028(String judul_3028) {
            this.judul_3028 = judul_3028;
        }

        public void setUrl_3028(String url_3028) {
            this.url_3028 = url_3028;
        }

}
```

Gambar 1

Kelas Website_2511533028 merupakan implementasi dari **Abstract Data Type (ADT)** yang digunakan untuk merepresentasikan satu data website dalam sistem browser history. Di dalam kelas ini dideklarasikan dua atribut utama yaitu judul_3028 dan url_3028 yang bertipe String, di mana atribut tersebut menyimpan informasi nama halaman website dan alamat URL-nya. Kedua atribut ini menggunakan modifier private, yang berarti tidak dapat diakses secara langsung dari luar kelas, sehingga menerapkan prinsip **enkapsulasi** untuk menjaga keamanan dan kontrol terhadap data.

Untuk menginisialisasi nilai dari atribut tersebut, kelas ini menyediakan sebuah **constructor** bernama Website_2511533028 yang menerima dua parameter yaitu judul dan URL. Constructor ini akan langsung mengisi nilai atribut saat objek dibuat, sehingga setiap objek website yang diinstansiasi sudah memiliki data yang lengkap sejak awal tanpa perlu diisi secara terpisah.

Selanjutnya, kelas ini menyediakan method **getter (selektor)** yaitu getJudul_3028() dan getUrl_3028() yang berfungsi untuk mengambil nilai dari atribut judul dan URL secara aman. Selain itu, terdapat juga method **setter (mutator)** yaitu setJudul_3028() dan setUrl_3028() yang digunakan untuk mengubah nilai atribut jika diperlukan setelah objek dibuat. Dengan adanya getter dan setter, akses terhadap atribut menjadi lebih terkontrol dan fleksibel tanpa melanggar prinsip enkapsulasi.

Secara keseluruhan, kelas Website_2511533028 berfungsi sebagai model data yang terstruktur untuk menyimpan informasi website, di mana setiap objek yang dibuat dari kelas ini akan mewakili satu halaman web dan dapat digunakan oleh kelas driver untuk dikelola dalam struktur data Stack sebagai bagian dari simulasi browser history.

2. Class Browser (Browser_2511533028)

```java
public class Browser_2511533028 {
    public static void main(String[] args) {
        Scanner input_3028 = new Scanner(System.in);
        Stack<Website_2511533028> history_3028 = new Stack<>();

        int pilihan_3028;

        do {
            System.out.println("\n=== Browser History NIM: 2511533028 ===");
            System.out.println("1. Kunjungi Website (Push)");
            System.out.println("2. Tombol Back (Pop)");
            System.out.println("3. Lihat Halaman Aktif (Peek)");
            System.out.println("4. Cek Status History");
            System.out.println("5. Keluar");
            System.out.print("Pilihan: ");
            pilihan_3028 = input_3028.nextInt();
            input_3028.nextLine();

            switch (pilihan_3028) {
                case 1:
                    System.out.print("Masukkan Judul: ");
                    String judul_3028 = input_3028.nextLine();

                    System.out.print("Masukkan Url: ");
                    String url_3028 = input_3028.nextLine();

                    Website_2511533028 web_3028 = new Website_2511533028(judul_3028, url_3028);
                    history_3028.push(web_3028);

                    System.out.println("Berhasil mengunjungi halaman!");
                    break;

                case 2:
                    if (!history_3028.isEmpty()) {
                        Website_2511533028 removed_3028 = history_3028.pop();
                        System.out.println("Kembali dari: " + removed_3028.getJudul_3028());
                    } else {
                        System.out.println("History kosong, tidak bisa kembali");
                    }
```

```java
                    break;

                case 3:
                    if (!history_3028.isEmpty()) {
                        Website_2511533028 current_3028 = history_3028.peek();
                        System.out.println("Halaman aktif:");
                        System.out.println("Judul: " + current_3028.getJudul_3028());
                        System.out.println("URL: " + current_3028.getUrl_3028());
                    } else {
                        System.out.println("Tidak ada halaman yang dibuka.");
                    }
                    break;

                case 4:
                    System.out.println("Jumlah history: " + history_3028.size());
                    if (history_3028.isEmpty()) {
                        System.out.println("History kosong.");
                    } else {
                        System.out.println("History tidak kosong.");
                    }
                    break;

                case 5:
                    System.out.println("Program selesai.");
                    break;

                default:
                    System.out.println("Pilihan tidak valid!");

            }
        } while (pilihan_3028 != 5);

        input_3028.close();
    }

}
```

Kelas Browser_2511533028 merupakan **kelas driver** yang berfungsi sebagai pusat kendali program untuk mensimulasikan browser history menggunakan struktur data **Stack** dengan prinsip **LIFO (Last In, First Out)**. Eksekusi program dimulai dari method main, yang menjadi titik awal jalannya program. Di dalamnya dibuat objek Scanner untuk menerima input dari pengguna, serta sebuah **Stack bertipe Website_2511533028**yang digunakan untuk menyimpan riwayat halaman website yang dikunjungi.

Program kemudian menggunakan variabel pilihan untuk menyimpan input menu dari pengguna dan membungkus seluruh proses dalam struktur perulangan **do-while**, sehingga menu akan terus ditampilkan sampai pengguna memilih keluar. Menu yang ditampilkan berisi beberapa fitur utama, dan setiap pilihan pengguna akan diproses menggunakan struktur **percabangan switch-case** untuk menentukan aksi yang sesuai.

Pada saat pengguna memilih menu kunjungi website (push), program akan meminta input berupa judul dan URL, kemudian membuat objek baru dari kelas Website_2511533028 dan menambahkannya ke dalam Stack menggunakan method .push(). Data yang baru dimasukkan akan berada di posisi paling atas Stack, sesuai dengan konsep LIFO.

Ketika pengguna memilih tombol back (pop), program akan mengecek terlebih dahulu apakah Stack kosong menggunakan .isEmpty() untuk menghindari error. Jika tidak kosong, elemen teratas Stack akan diambil dan dihapus menggunakan .pop(), kemudian ditampilkan judul halaman yang telah

ditutup. Hal ini menunjukkan bahwa halaman terakhir yang dibuka adalah yang pertama kali dikeluarkan.

Pada fitur lihat halaman aktif (peek), program kembali mengecek apakah Stack kosong. Jika tidak, maka elemen paling atas Stack akan diambil menggunakan .peek() tanpa menghapusnya, lalu ditampilkan informasi judul dan URL-nya. Fitur ini menggambarkan halaman yang sedang aktif dibuka pada browser.

Untuk fitur cek status history, program menampilkan jumlah data yang tersimpan dalam Stack menggunakan method .size(), serta mengecek apakah Stack kosong dengan .isEmpty(), sehingga pengguna dapat mengetahui kondisi riwayat browsing saat ini.

Terakhir, jika pengguna memilih keluar, perulangan akan dihentikan dan program selesai dijalankan. Secara keseluruhan, kelas ini menggabungkan penggunaan **input/output, perulangan, percabangan, dan struktur data Stack**, sehingga seluruh fitur seperti push, pop, peek, dan pengecekan status dapat berjalan dengan baik sesuai konsep dasar Stack dalam simulasi browser history.

OUTPUT JIKA BERISI :

```
=== Browser History NIM: 2511533028 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 1
Masukkan Judul: SSO UNAND
Masukkan Url: https://sso.unand.ac.id/login
Berhasil mengunjungi halaman!
```

```
=== Browser History NIM: 2511533028 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 2
Kembali dari: SSO UNAND
```

```
=== Browser History NIM: 2511533028 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 4
Jumlah history: 0
History kosong.
```

OUTPUT JIKA KOSONG :

```
=== Browser History NIM: 2511533028 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 3
Tidak ada halaman yang dibuka.
```

```
=== Browser History NIM: 2511533028 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 4
Jumlah history: 0
History kosong.
```